## MODULE-4

# TREES

**TOPICS:-** Terminology, Binary Trees, Properties of Binary trees, Array and linked Representation of Binary Trees, Binary Tree Traversals - Inorder, postorder, preorder; Additional Binary tree operations. Threaded binary trees, Binary Search Trees – Definition, Insertion, Deletion, Traversal, Searching, Application of Trees-Evaluation of Expression, Programming Examples.
Text 1: Ch 5: 5.1 –5.5, 5.7
Text 2: Ch 7: 7.1 – 7.9

## INTRODUCTION

A tree is non-linear and a hierarchical data structure consisting of a collection of nodes such that each node of the tree stores a value, a list of references to nodes (the "children").

### 4.1 DEFINITION

**Tree** is a finite set of one or more nodes such that

1) There is a specially designated node called root.

2) Remaining nodes are partitioned into disjoint sets T1, T2. . . Tn where each of these are called subtrees of root. (As shown in below figure1)

(Or)

A **Tree** is a set of nodes that either: is empty or has a designated node, called the root, from which hierarchically descend zero or more subtrees, which are also trees. If a tree is not empty, the first node is called the root .The indegreee of root node is zero. With the exception of the root, all the nodes in the tree must have an indegreee exactly one and the out degree of zero, one, or more.

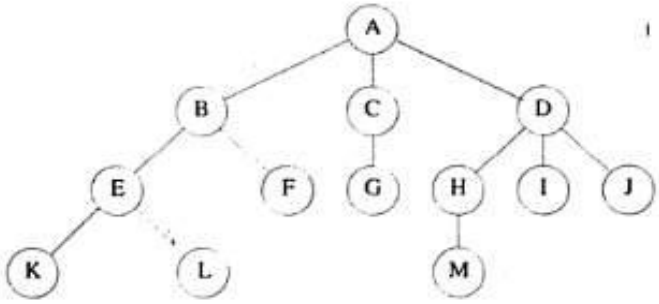



**Figure 1: Sample Tree**

**A tree** consists of a finite set of elements called **nodes** and a finite set of directed lines called **branches** that connect the nodes. The number of branches associated with a node is the degree of the node. When the branch is directed toward the node, it is indegree branch. When the branch is directed away from the node it is an outdegree branch.

## 4.2 TERMINOLOGIES OF TREES

- **Node:** contains item of information & links to other nodes.
- **Root:** A node with an indegree zero i.e. a node with no parent is called root; A non-empty tree has exactly one Root.
- **Degree:** Number of subtrees of a node. For e.g., degree of A=3; degree of C=1
- **Degree of a tree** is the maximum of the degree of the nodes in the tree. Degree of given tree=3.
- **Leaf:** Any nodes with an outdegree of zero i.e. a node with no successors or children's. For e.g., K, L, F, G, M, I, J
- **Internal Node:** A node that is not a root or a leaf is known as an internal node. For e.g., B, E, F, C, H, I, J
- **Parent and Child:** The subtrees of a node A are the children of A. A is the parent of its children (OR) a node is a parent if it has successor nodes i.e. if it has an outdegree greater than zero. Conversely, a node with predecessor is a child. For e.g., children of D are H, I and J. Parent of D is A.
- **Siblings/Brothers/Sisters:** Children of same parent are called siblings. For e.g., H, I and J are siblings.
- **Ancestor:** An ancestor is any node in the path from the root to the node. For e.g., ancestors of M are A, D and H.
- **Descendent:** A descendent is any node in the path below the parent node i.e. all nodes in the paths from a given node to a leaf are descendents of that node.
- **Path:** a sequence of nodes in which each node is adjacent to the next one. Every node in the tree can be reached by following a unique path starting from the root. the length of a path is the number of edges in the path, or 1 less than the number of nodes in it.
- **Level:** the level of a node is its distance from the root. If a node is at level 'l', then its children are at level 'l+1'.
- **Height or depth of a tree** is defined as maximum level of any node in the tree. For e.g.,

Height of given tree = 4.

- A **Sub tree** is any connected structure below the root.
- A **tree** is a set of nodes that either: Is empty , or Has a designated node, called the root, from which hierarchically descend zero or more subtrees, which are also trees.

## 4.3 BINARY SEARCH TREES

Binary search tree (BST), which may sometimes also be called an ordered or sorted binary tree, is a node-based binary tree data structure.

BST Properties:

- The left subtree of a node contains only nodes with keys less than the node's key.
- The right subtree of a node contains only nodes with keys greater than the node's key.
- Both the left and right subtrees must also be binary search trees.

***Note: Every node in the tree should satisfy this condition.***

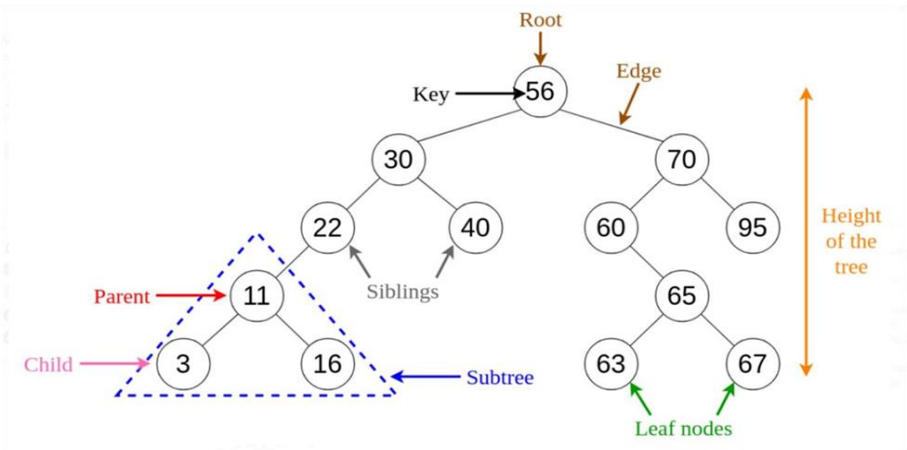



**Figure 2: Binary Search Tree**

### 4.3.1 Construction of BST

If the tree is empty

- Insert the new key in the root node

else if the root's key matches the new key

- Skip insertion

else if the new key is smaller than root's key

- Insert the new key in the left subtree

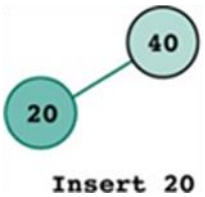
else

- Insert the new key in the right subtree

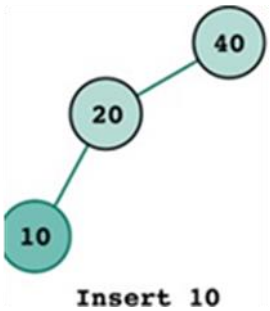
**Example 1: Assume 40, 20, 10, 50, 65, 45, 30 are inserted in order for BST construction.**

**Step 1:** Initially root is NULL, there node 40 becomes the root node

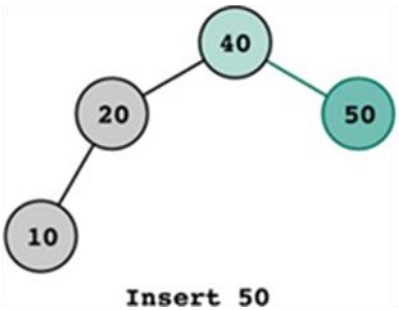
**Step 2:** Node 20 is lesser than the root node 40, therefore node 20 is inserted as the left child of node 40



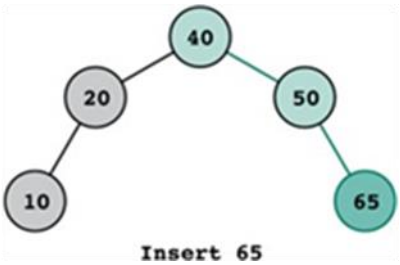

**Step 3:** Node 10 is lesser than node 40, then insertion is done in the left subtree of node 40. Node 10 is compared with node 20. As node 10 is lesser it is inserted as the left child of node 20.



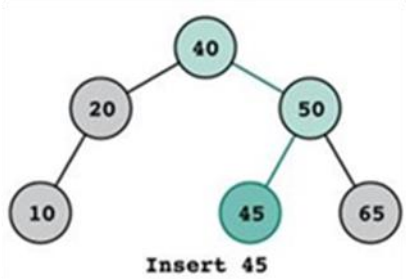

**Step 4:** Node 50 is greater than node 40, so it is inserted to the right of node 40



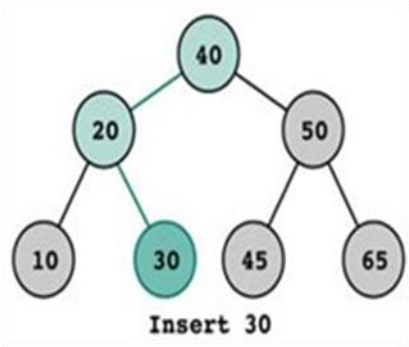

**Step 5:** Node 65 is greater than node 40, compare node 65 with the right subtree of the node 40. Node 65 is compared with node 50. As node 65 is greater it is inserted to the right subtree of node 50.



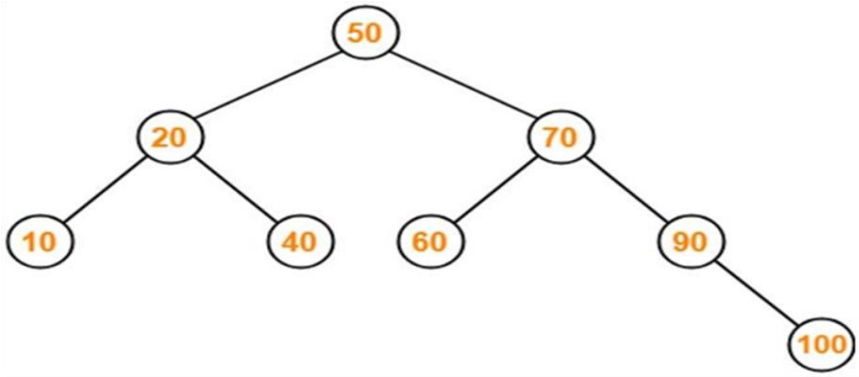

**Step 6:** Node 45 is greater than root node 40, so recursively compare in the right subtree of node 40. Node 45 is compared with node 50 and inserted as the left child of node 50



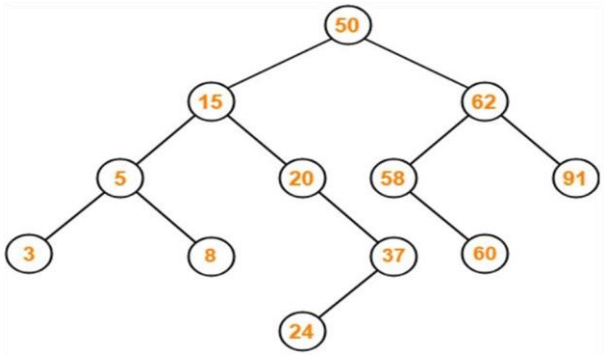

**Step 7:** Similarly node 30 is compared with node 40, then compared with node 20 and finally inserted as the right child of node 20




**Example 2: Construct BST for the given numbers- 50, 70, 60, 20, 90, 10, 40, 100**



**Example 3: Construct BST for the given numbers- 50, 15, 62, 5, 20, 58, 91, 3, 8, 37, 60, 24**




### 4.3.2 Implementation of Binary Search Trees

Binary Search Trees can be implemented using Linked list and arrays. This section starts with the linked list implementation and later array implementation will be discussed.

**Linked list implementation:** Insertion and deletion of nodes from middle of a tree requires movement of potentially many nodes to reflect the change in level number of these nodes. These problems can be overcome easily through the use of a linked representation Doubly linked list is used to represent each node in a BST for linked list implementation.

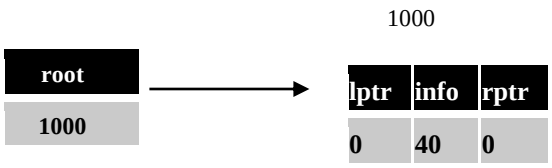
Here each node has 3 fields.

- Info is used to store the actual data.
- lptr is used to store the address of left sub tree.
- rptr is used to store the address of right sub tree

*Note: If lptr is NULL then there is no left sub tree, if rptr is NULL then there is no right sub tree.*

**Structure of node:**

```
struct bst
{
        int info;
         struct bst *lptr,*rptr;
};
typedef struct bst node;
```



**Insertion into BST: Assume data 40, 20, 10, 50, 65, 45, 30 to be inserted into BST**

```
node* insert(node *root)
{
        node *new1, *cur = root, *prev= NULL;
        new1 = (node *)malloc(sizeof(node));
        printf("\nEnter The Element ");
        scanf("%d",&new1->item);
         new1->lptr = new1->rptr = NULL;
         if (root == NULL)
              return new1;
         while(cur != NULL)
         {
                  prev = cur;
                  cur = new1->item < cur->item ?
                  cur->lptr : cur->rptr;
          }
          if (new1->item < prev->item)
                  prev->lptr = new1;
          else
                  prev->rptr = new1;
  return root;
}
```

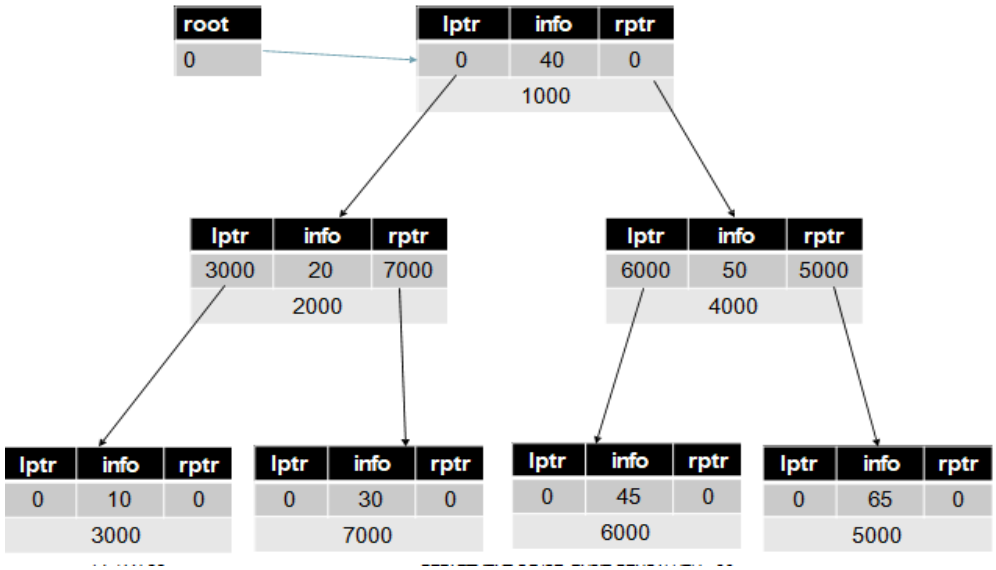



**Figure 3: Linked list implementation of sample BST**

### 4.3.3 $T^3$ : Tree Traversal Techniques

The order in which the nodes of a linear list are visited in a traversal is clearly from first to last. However, there is no such "natural" linear order for the nodes of a tree. Thus, different orderings are used for traversal in different cases. Traversing a BT/BST involves

- Visiting the root node
- Traversing its left and right sub trees.

We designate the task of visiting the root node as Rt, traversing the left subtree as Ls and traversing the right subtree as Rs. Different tree traversal techniques are :

- **In order Traversal -- Ls Rt Rs**
- **Preorder Traversal -- Rt Ls Rs**
- **Post order Traversal -- Ls Rs Rt**

**Inorder Traversal**: If the root is visited *between* traversing the subtree, it is called the preorder traversal.

- Traverse the Left sub tree in inorder recursively.
- Process the root node
- Traverse the Right sub tree in inorder recursively

**Implementaion of Inorder traversal:**

```
void inorder(node *root)
{
        if(root != NULL)
        {
                inorder(root->lptr);
                printf("%d ",root->info);
                inorder(root->rptr);
        }
}
```

Inorder traversal for the tree given in figure 3:

| | | | | | | |
|---|---|---|---|---|---|---|
| 10 | 20 | 30 | 40 | 45 | 50 | 65 |

**Preorder Traversal:** If the root is visited before traversing the subtree, it is called the preorder traversal.

- Process the root node
- Traverse the Left sub tree in preorder recursively.
- Traverse the Right sub tree in preorder recursively

Implementation of Preorder Traversal:

```
void  preorder(node *root)
{
        if(root  != NULL)
        {
                printf("%d ",root->info);
                preorder(root->lptr);
                preorder(root->rptr);
        }
}
```

Preorder traversal for the tree given in figure 3:

| | | | | | | |
|---|---|---|---|---|---|---|
| 40 | 20 | 10 | 30 | 50 | 45 | 65 |

**Post order Traversal:** If the root is visited after traversing the subtrees, it is called post order traversal.

- Traverse the Left sub tree in postorder recursively.
- Traverse the Right sub tree in postorder recursively
- Process the root node

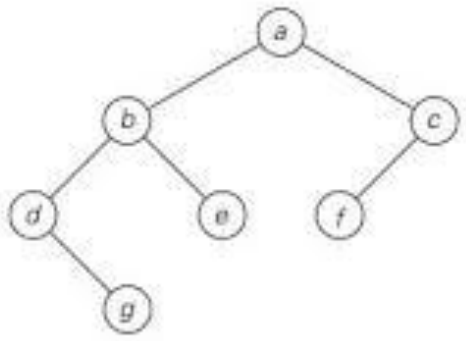
Implementation of postorder traversal:

```
void postorder(node *root)
{
        if( root != NULL )
        {
                postorder(root->lptr);
                postorder(root->rptr);
                 printf("%d ",root->info);
        }
}
```

Postorder traversal for the tree given in figure 3:

| | | | | | | |
|---|---|---|---|---|---|---|
| 10 | 30 | 20 | 45 | 65 | 50 | 40 |

**Example 2: Write down tree traversals for the below given tree**




**Preorder:** ***a b d g e c f***

**Inorder*:*** ***d g b e a f c***

**Postorder:** ***g d e b f c a'***

**C Function to search a node in BST:**

```
void search(node *root, int key)
{
        if(root != NULL)
        {
                if( root->info == key)
                {
                        printf("search is success");
                        return (1);
                }
                if(key > root->info )
                        search(root->rptr,key);
                else
                        search(root->lptr,key);
        }
        return (0);
}
```

**C Function to print all leaves in BST:**

```
void print( node *root)
{
        if root != NULL)
        {
                print( root->lptr);
```

```
                if(root->lptr == NULL && root->rptr == NULL)
                        printf("%d ",root->info);
                printf( root->rptr);
        }
}
```

**C Function to print all leaves in BST:**

```
void print( node *root)
{
        if root != NULL)
        {
                print( root->lptr);
                if(root->lptr != NULL || root->rptr != NULL)
                        printf("%d ",root->info);
                printf( root->rptr);
        }
}
```

**C Function to print height of a BST:**

```
int height( node *root)
{       if root == NULL)
                return(0);
        else
                return( 1 + max(height(root->lptr), height( root->rptr) );
}
```

**C Function to find minimum value of a BST:**

```
node * Minimum( node *root)
{
        node *cur =     root;
        if (root == NULL)
                return NULL;
        while( cur->    lptr !=NULL)
                cur = cur->lptr;
        return cur;
}
```

**C Function to find maximum value of a BST:**

```
node * Maximum( node *root)
{
        node *cur =     root;
```

```
        if (root == NULL)
                return NULL;
        while( cur->    rptr !=NULL)
                cur = cur->rptr;
        return cur;  }
```

**C Function to get duplicate copy of a BST:**

```
node*  copy( node *root)
{
        node *new;
        if (root == NULL)
                return NULL;
        new = (node *) malloc(size of(node));
        new -> info = root -> info;
        new->lptr= copy(root->lptr);
        new ->rptr= copy(root->rptr);
        return new;
}
```

**C Function to print height  of a BST:**

```
int height( node *root)
{
        if root == NULL)
                return(0);
        else
                return( 1 + max(height(root->lptr), height( root->rptr) );
}
```

### 4.3.4 Deleting a node from BST

Algorithm for Deleting a node in BST

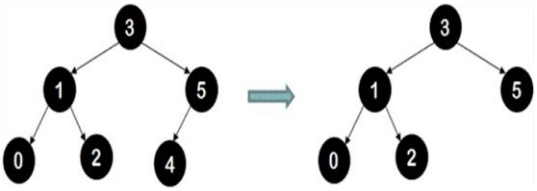
1. Input the data of the node to be deleted.
2. If root is NULL
   BST is empty and stop.
3. If data < root->item then
   Call delete() in left subtree and stop.
4. If data > root->item then
   Call delete() in right subtree and stop.
5. If the node is a leaf node, delete the node directly and stop.
6. If the node has one child, copy the child to the node to be deleted and delete the child node and stop.
7. If the node has two children
   I. Copy the contents of the inorder successor to to be deleted
   II. Delete the inorder successor and stop.

**Case 1:**

```
if (root == NULL)
{
        printf("tree is empty\n");
        return NULL;
}
```

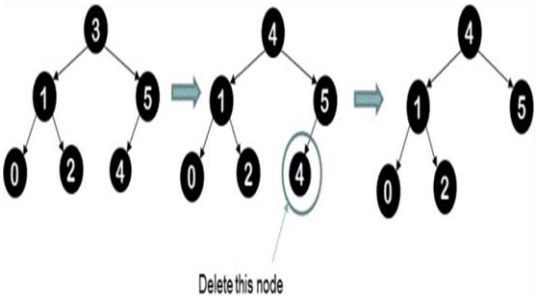
**Case 2: Data is in the left subtree**

```
if (data < root->item)
{
        root->lptr = Delete(root->lptr, data);
        return(root);
}
```



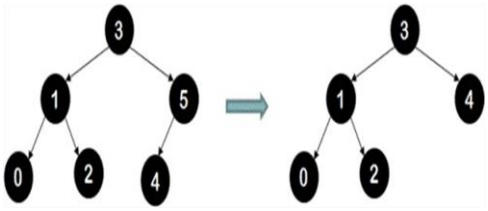
**Case3: Data is in the right subtree**

```
if (data > root->item)
{
        root->rptr = Delete(root->rptr, data);
        return(root);
}
```




**Case 4: Data is present but no children**

```
if (root->lptr == NULL && root->rptr == NULL)

{
        printf("deleted data %d",root->item);
        free(root);
        root = NULL;
         return(root);
}
```

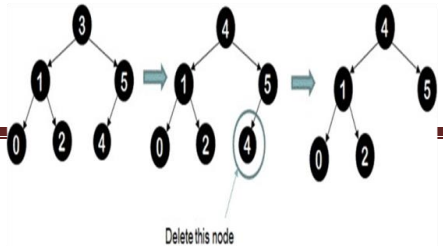


**Case 5 : data is present but no right subtree**

```
if (root->rptr == NULL)
{       temp = root; // save current node as a backup
        root     = root->lptr;
        printf("deleted data %d",temp->item);
        free(temp);
         return(root);
}
```

**Case 6: Both left and right subtree**

```
min = FindMin(root->rptr);
root->item = min;
root->rptr = Delete(root->rptr, min);
```



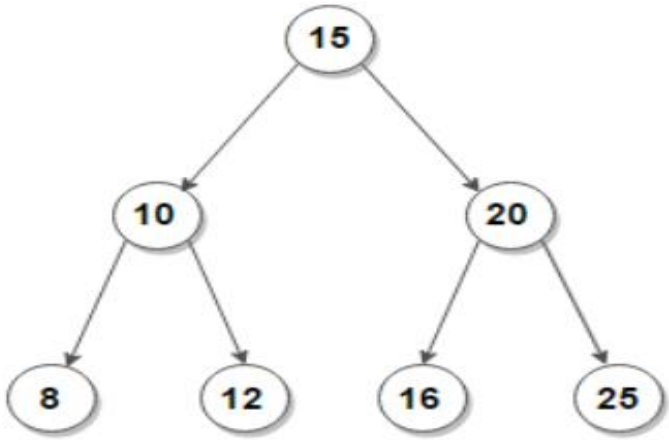
**C Function to delete a node from BST:**

```
node* Delete(node *root, int data)
{
        node *temp;    int min;
        if (root == NULL)
        {
                printf("tree is empty\n");
                return NULL;
        }
        // data is in the left sub tree.
        if (data  <  root->item)
        {
                root->lptr = Delete(root->lptr, data);
                return(root);
        }
        // data is in the right sub tree.
        if (data  >  root->item)
        {
                root->rptr = Delete(root->rptr, data);
                return(root);
        }
          // data is present but no children
        if (root->lptr == NULL && root->rptr == NULL)
        {
                printf("deleted data %d",root->item);
                free(root);
                 root = NULL;
                return(root);
        }

         // data is present but no right subtree
        if (root->rptr == NULL)
        {
                temp = root->lptr;
                printf("deleted data %d",root->item);
               free(root);
                 return(temp);
        }
         // data is present but no left subtree
         if (root->lptr == NULL)
         {
                temp = root->rptr;
                printf("deleted info %d",root->item);
```

```
            free(root); return(temp);
        }
        // If both left and right subtree are present, find the min element
        // in right subtree and place it in root node and call delete function
        //for right subtree.
         min = FindMin(root->rptr);
         root->item = min;
         root->rptr = Delete(root->rptr, min);
        return(root);

}
```

**4.3.5 Array Implementation of Binary Search Tree:**

A one-dimensional array can be used to store nodes of binary search tree. Root node starts from index 0. Consider the tree shown below, array representation of this tree is given in figure 5.




| a | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | … | 19 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | 15 | 10 | 20 | 8 | 12 | 16 | 25 | | | | | | | |

**Figure 5: Array representation of above binary search tree**

```
#include<stdio.h>
#include<stdlib.h>
#define SIZE 20

void insert(int item, int a[])
```

```
{
        int i =0;
         while( i  < SIZE && a[i] != 0)
         {
                if ( item < a[i])
                        i = 2 * i +1;
                   else
                        i = 2 * i +2;
         }
         a[i] = item;
}

void inorder(int a[], int i)
{
        if(a[i])
        {
                inorder(a,2*i+1);
                printf("%d ",a[i]);
                inorder(a,2*i+2);
        }
}

void preorder(int a[], int i)
{
         if(a[i])
        {
                printf("%d ", a[i]);
                preorder(a, 2 * i +1);
                preorder(a, 2 * i +2);
         }
}




void postorder(int a[], int i)
{
        if(a[i])
        {
                postorder(a, 2 * i +1);
                postorder(a, 2 * i +2);
                 printf("%d ", a[i]);
         }
}

void main()
{
        int a[20], item, ch, i;
         for(i = 0; i< SIZE; i++)        a[i] = 0;
```

```
        for(;;)
         {
                printf("1. Insert. 2. Inorder 3. Preorder 4. Postorder 5. Exit\n");
                 printf("Enetr your choice\n");
                 scanf("%d",&ch);
                  switch(ch)
                  {
                        case 1: printf("Enter the Item\n");
                                 scanf("%d",&item);
                                 insert(item,a); break;
                        case 2: if( a[0] == 0)
                                        printf("list is empty\n");
                                 else
                                 {
                                        printf("Inorder is ");
                                         inorder(a, 0);
                                 }
                                 break;

                        case 3:  if( a[0] == 0)
                                        printf("list is empty\n");
                                 else
                                 {
                                        printf("Preorder is");
                                        preorder(a,0);
                                 }
                                 break;
                        case 4: if( a[0] == 0)
                                        printf("list is empty\n");
                                  else
                                  {
                                        printf("Postorder is");
                                        postorder(a,0);
                                    }
                                 break;
                        default: printf("invalid Choice\n");  exit(0);
                  }
        }
}
```

## 4.4 Binary Trees

A **Binary tree** is a finite set of elements that is either empty or is partitioned into two disjoint subsets. The **first subset** contains a single element called the **root of the tree**. The **other two subsets** are themselves binary trees, called the **left and right subtrees of the original tree**. A left or right sub tree can be empty. Each element of a binary tree is called a node of the tree as

illustrated in figure 6.

(OR)

A **Binary tree** is a tree in which no node can have more than two subtrees. The maximum outdegree for a node is two. In other words, a node can have zero, one or two subtrees. These subtrees are designated as the left sub tree and right sub tree.

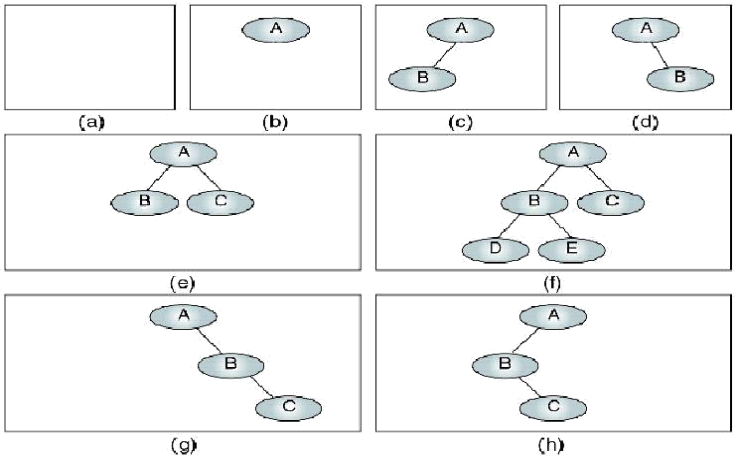



Figure 6: Various Binary Tree Examples

### 4.4.1 ADT of Binary Tree

```
structure Binary_Tree(abbreviated BinTree) is
objects: a finite set of nodes either empty or consisting of a root node, left Binary_Tree,
and right Binary_Tree.
functions:
for all bt, bt1, bt2 ∈ BinTree, item ∈ element
Bintree Create()::= creates an empty binary tree
Boolean IsEmpty(bt)::= if (bt==empty binary tree)
                            return TRUE
                        else
                            return FALSE
BinTree MakeBT(bt1, item, bt2)::= return a binary tree whose left subtree is bt1, whose
                                  right subtree is bt2, and whose root node contains the data item
Bintree Lchild(bt)::= if (IsEmpty(bt))
                          return error
                      else
                          return the left subtree of bt
element Data(bt)::= if (IsEmpty(bt))
                        return error
                    else
                        return the data in the root node of bt
Bintree Rchild(bt)::= if (IsEmpty(bt))
                        return error
                    else
                        return the right subtree of bt
```

### 4.4.2 PROPERTIES OF BINARY TREES

- The **maximum number of nodes on level 'i'** of a binary tree is **$2^{i-1}$, i >=1.**
  **(For e.g. maximum number of nodes on level 4=$2^{4-1}=2^3$=8).**

- The **maximum number of nodes in a binary tree of depth 'k'** is **$2^k$-1, k>=1.**
  **(For e.g. maximum number of nodes with depth 4=$2^4$-1=16-1=15).**

- **Relation between number of leaf nodes and degree-2 nodes** : For any non-empty binary tree 'T', if $n_0$ is the number of leaf nodes and $n_2$ the number of nodes of degree 2, then **$n_0=n_2+1$.**

- **A full binary tree of depth k is a binary tree of depth k having $2^k$-1 nodes, k>=0.**

- **A binary tree with n nodes and depth k is complete** iff its nodes correspond to the nodes numbered from 1 to n in the full binary tree of depth k.

- **Minimum number of nodes binary trees:** Given a height of the binary tree, H, the minimum number of nodes in the tree is equal to height of tree.

- **Height of Binary Trees:** If the given binary tree has N nodes in a binary tree, the maximum height is given by Hmax=N. If the given binary tree has N nodes in a binary

tree, the minimum Height Is Given By Hmin=Log2n+1

- **Level of binary trees:** The level of a node in a binary tree is defined as follows. The root has level 0, and the level of any other node in the tree is one more than the level of its father.

- **Depth of binary trees:** The depth or Height of a binary tree is the maximum level of any leaf in the tree. By definition the height of any empty tree is -1. This equals the length of the longest path from the root to any leaf.

**Differentiate between a binary tree and a tree.**

1. There is no tree having zero nodes, but there is an empty binary tree.
2. In a binary tree, we distinguish between the orders of the children while in a tree we do not.

### 4.4.3 TYPES OF BINARY TREE

1) **Skewed tree** is a tree consisting of only left sub tree or only right sub tree (Figure 7a).

2) If every non leaf node in a binary tree has nonempty left and right subtrees, the tree is termed a **Strictly binary tree**. A strictly binary tree with n leaves always contains 2n-1 nodes as illustrated in figure 7b.

2) **Complete binary tree** is a binary tree of depth k having $2^k$-1 nodes, k>=1 (Figure 8).

3) An **Almost complete binary tree** is a binary tree that satisfies the following 2 properties- (Figure 9).

- All the levels are completely filled except possibly the last level.
- The last level must be strictly filled from left to right.

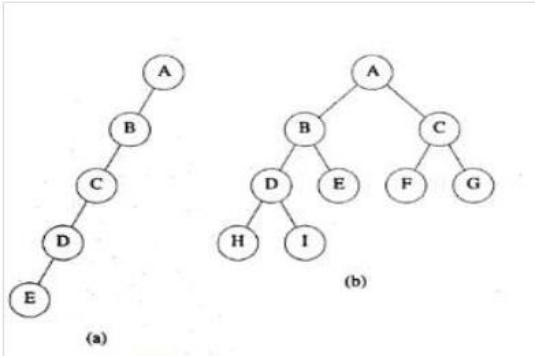


**Figure 7(a): skewed tree**

**7(b).Strictly binary tree**

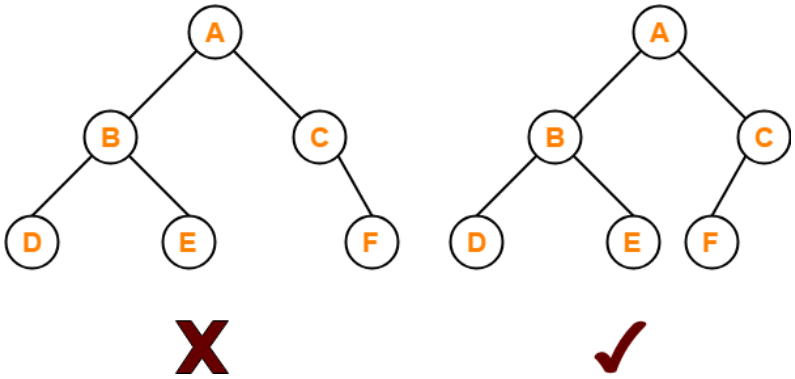



**Figure 8: Full binary tree representation of depth 4 with sequential node numbering**

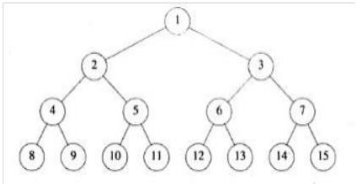



**Figure 9: Complete binary tree representation**

## 4. 5 Application of Trees: EXPRESSION TREES

Expression tree is a binary tree, because all of the operations are binary. It is also possible for a node to have only one child, as is the case with the unary minus operator. The leaves of an expression tree are operands, such as constants or variable names, and the other (non leaf) nodes contain operators.

Once an expression tree is constructed, we can traverse it in three ways:

- Inorder Traversal

- Preorder Traversal
- Postorder Traversal

Figure 10 shows some more expression trees that represent arithmetic expressions given in infix form.

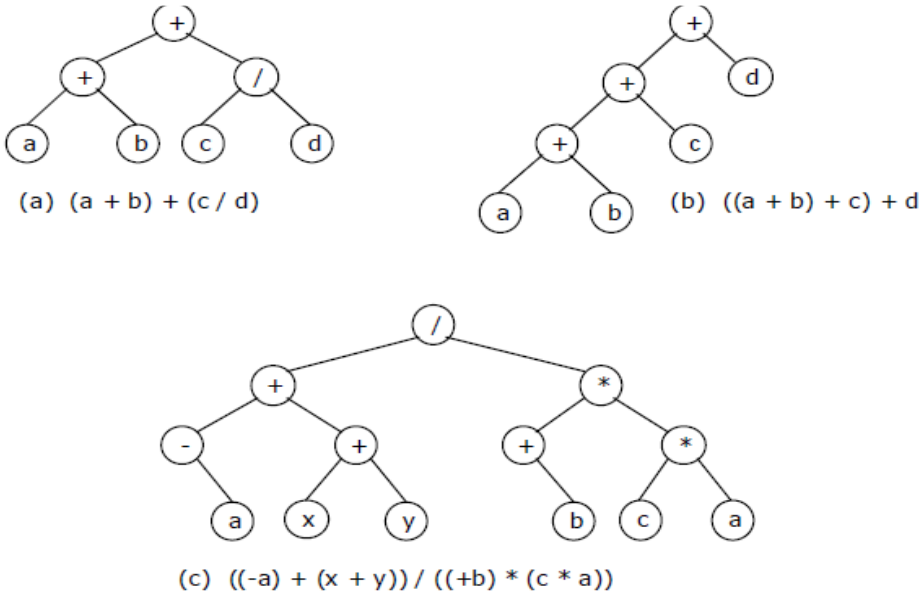



**Figure 10: Expression trees**

### 4.5.1 Construction of expression tree

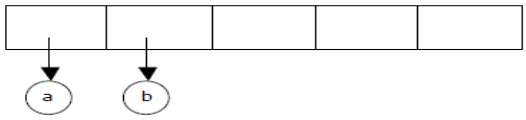
An expression tree can be generated for the infix and postfix expressions. An algorithm to convert a postfix expression into an expression tree is as follows:

- Read the expression from left to right and one symbol at a time.
- If the symbol is an **operand,** we create a one-node tree and push a pointer to it onto a stack.
- If the symbol is an **operator,** we pop pointers to two trees T1 and T2 from the stack (T1 is popped first) and form a new tree whose root is the operator and whose left and right children point to T2 and T1 respectively. A pointer to this new tree is then pushed onto the stack.

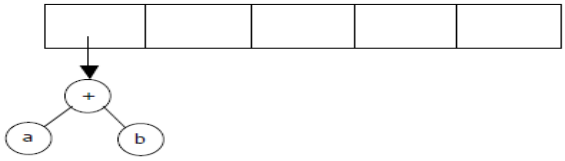
Construct an expression tree for the postfix expression: a b + c d e + * *

**Solution:**

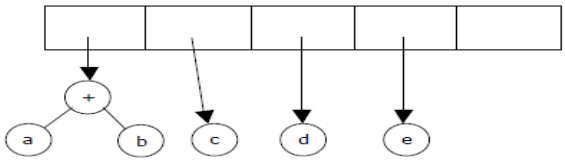
The first two symbols are operands, so we create one-node trees and push pointers to them onto a stack.



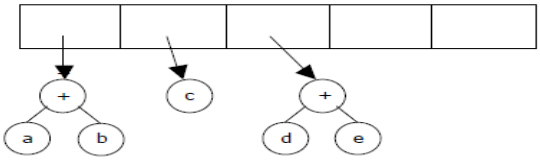

Next, a '+' is read, so two pointers to trees are popped, a new tree is formed, and a pointer to it is pushed onto the stack.



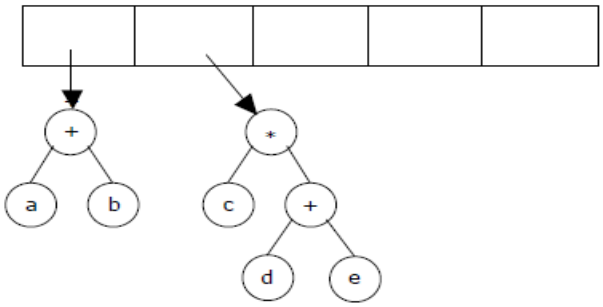

Next, c, d, and e are read, and for each one-node tree is created and a pointer to the corresponding tree is pushed onto the stack.



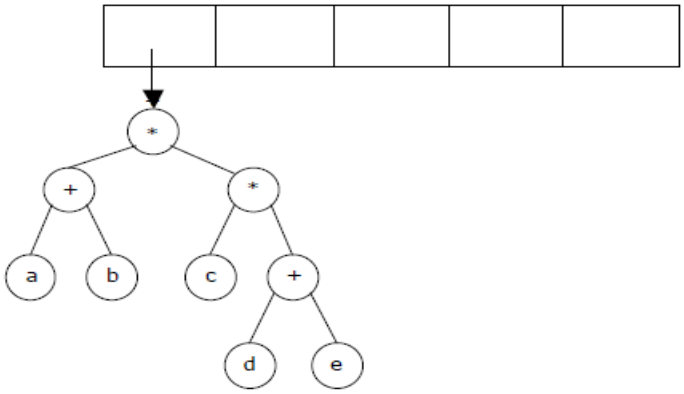

Now a '+' is read, so two trees are merged.



Continuing, a '*' is read, so we pop two tree pointers and form a new tree with a '*' as root.




Finally, the last symbol is read, two trees are merged, and a pointer to the final tree is left on the stack.




**For the above tree:**

Inorder form of the expression: a + b * c * d + e

Preorder form of the expression: * + a b * c + d e

Postorder form of the expression: a b + c d e + * *

***C program to construct Expression tree***

```
struct bst
{  char info;
   struct bst *lptr, *rptr;
};
typedef struct bst node;
node* create(char post[])
{  node *new1, *st[20];
   char sym; int i, k =0;
```

```
for(i=0; sym = post[i] !='\0'; i++)
{
   new1 = (node *)malloc(sizeof(node));
   new1->info = sym;
   new1->lptr = new1->rptr = NULL;
     if (isalnum(sym))
            st[k++] = new1;
  else
  {   new1->rptr = st[--k];
      new1->lptr =st[--k];
      st[k++] = new1;
  }
}
return st[--k];
}
float eval(node *root)
{
             float num;
    switch(root->info)
    { case '+' : return(eval(root->lptr) + eval(root->rptr)); break;
      case '-' : return(eval(root->lptr) -  eval(root->rptr)); break;
      case '*' : return(eval(root->lptr)  * eval(root->rptr)); break;
      case '/' : return(eval(root->lptr) / eval(root->rptr)); break;
      default: if(isalpha(root->info))
               { printf("%c ", root->info);
                       scanf("%f",&num);
                       return(num);
                     }
    }
}
```

```
void main()
{
        char postfix[20]; float res;
        node *root = NULL;
    printf("\n Enter Postfix expression\n");
    scanf("%s",postfix);
    root = create(postfix);
    res = eval(root);
    printf(" Result = %f\n",res);
}
```

### *4.5.2 Building binary trees from traversal pairs*

Constructing Binary tree using any 2 Traversals.

- Inorder and preorder
- Inorder and postorder
- Inorder and level order

**Construct a binary tree from a given preorder and inorder sequence:**

**Preorder: A B D G C E H I F**

**Inorfer: D G B A H E I C G**

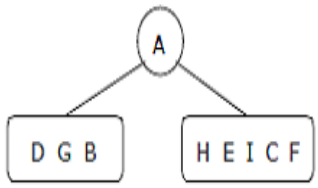
**Step 1:** From preorder sequence **A** B D G C E H I F, the root is A

From Inorder sequence D G B **A** H E I C G, we get the left and right sub trees:

Left sub tree is D G B

Right subtree is H E I C F

The Binary tree upto this point looks like:



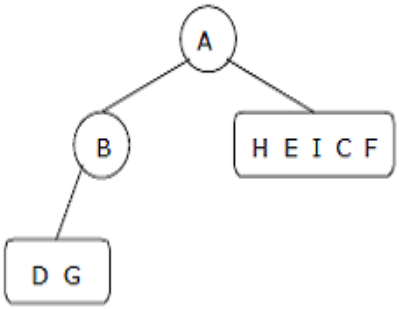
**Step 2:** To find the root, left and right sub trees for D G B:

From the preorder sequence B D G, the root of tree is :B

From the inorder sequence D G B, we can find that D and G are to the left of B

Binary tree upto this point looks like

The Binary tree upto this point looks like:

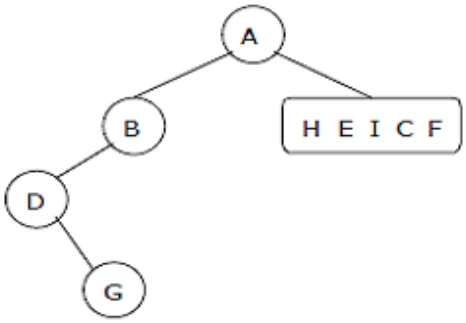



**Step 3:** T o find the root, left and right subtrees for D G:

From the preorder sequence **D** G, the root of the tree is : D

From the inorder sequence **D** G, we can find that there is no left node to D and G is at the right od D

The Binary tree upto this point looks like:

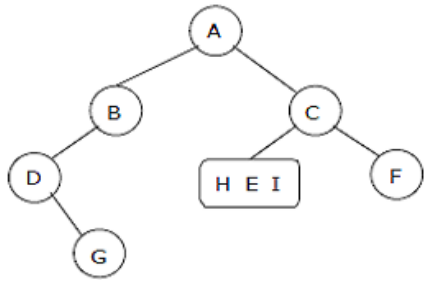



**Step 4:** To find the root, left and right subtrees for H E I C F:

From the preorder sequence **C** E H I F, the root of the left sub tree is : C

From the inorder sequence H E I **C** F, we can find that H E I are at the left of C and F is at the right of C

The Binary tree upto this point looks like:

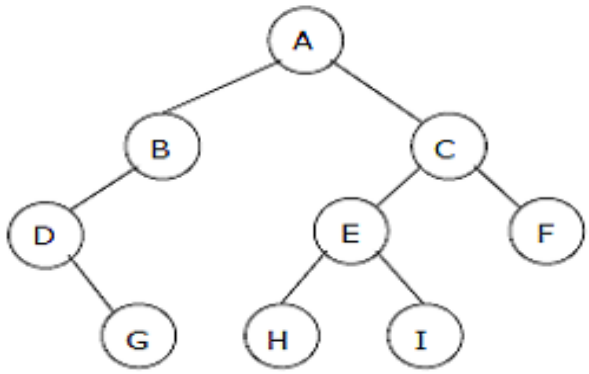



**Step 5:** To find the root, left and right sub trees for H E I:

From the preorder sequence **E** H I, the root of the tree is E

From the inorder sequence H **E** I, we can find that H is at the left of E and I is at the right of E

The Binary tree upto this point looks like:




Exercise: 1

1. Construct a Binary tree for the following traversals
   a) Inorder sequence: D B E A F C
   b) Preorder sequence: A B D E C F
2. Construct a binary tree for the following traversals
   a) Inorder Traversal : { 4, 2, 1, 7, 5, 8, 3, 6 }
   b) Preorder Traversal: { 1, 2, 4, 3, 5, 7, 8, 6 }

### *4.6 Different User representation for trees*

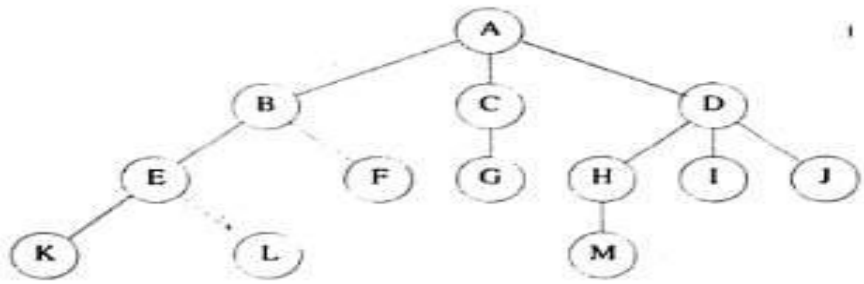



**Figure 11: Tree**

There are **three different user representations for trees.**

1) List representation

2) Left-child right-sibling representation

3) Degree-two tree representation (Binary Tree)

**1) List representation**

This user format is the **parenthetical listing**. This format is used with algebraic expressions. When a tree is represented in parenthetical notation, each **open parenthesis indicates the start of a new level, each closing parenthesis completes the current level, and each closing parenthesis completes the current level and moves up one level in the tree.**

Consider the tree shown in the figure 11. Its parenthetical notation is The tree can be drawn as a list: **(A(B(E(K,L),F),C(G),D(H(M),I,J))).**

For a tree of degree 'k', we can use the node-structure as shown(Fig. 12).

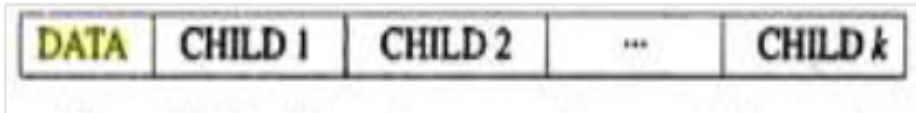



**Figure 12: Node structure with degree k**

The information in the root node comes first, followed by a list of subtrees of that node. Each tree-node can be represented by a memory-node that has fields for data & pointers to children of tree-node (Figure 13).

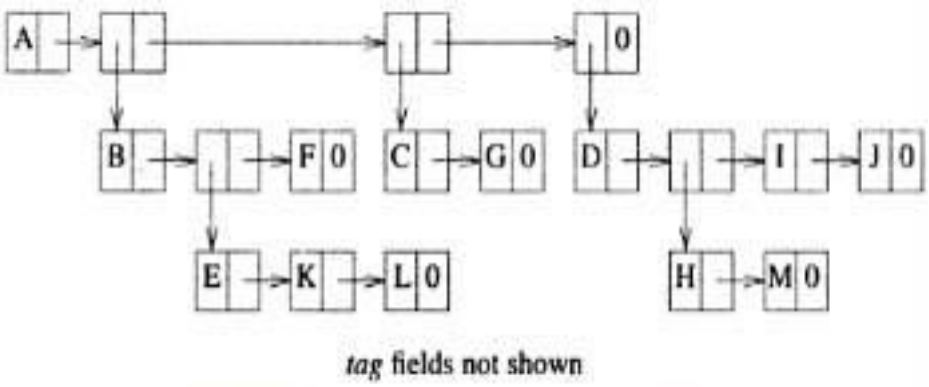



**Figure 13: List Representation of Tree**

**Property of general tree:** if T is a k-ary tree with n nodes, each having a fixed size as in figure 3, then n(k-1)+1 of the nk child fields are 0,n>=1.

**2) Left child-right sibling representation**

Figure 14 shows the node-structure used in left child-right sibling representation.

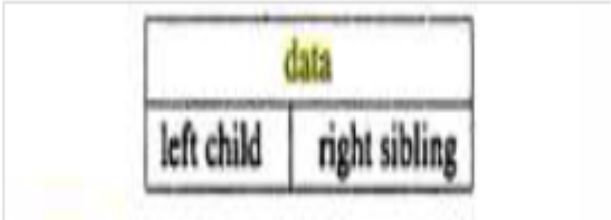



**Figure 14: Left Child-Right Sibling Node**

To convert the tree of figure 11 into this representation, every node must note it has at most one leftmost child and at most one closest right sibling.

For example, the leftmost child of A is B, and the leftmost child of D is H. The closest right sibling of B is C, and the closest right sibling of H is I. The Left-child field of each node points to its leftmost child (if any), and right-sibling field points to the closest right sibling (if any). Figure 15 shows the left child right sibling representation.

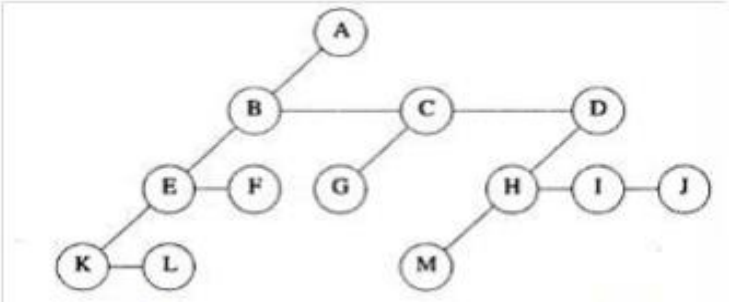



**Figure 15: Left Child Right Sibling Representation.**

**3) Degree-two tree representation**

To obtain the degree two tree representation of a tree, simply rotate the right-sibling pointers in a left child-right sibling tree clockwise by 45 degrees. This gives the degree two tree displayed in figure 16, In this representation, it refer to 2 children of a node as left & right children. Left child-right child trees are also known as binary trees.

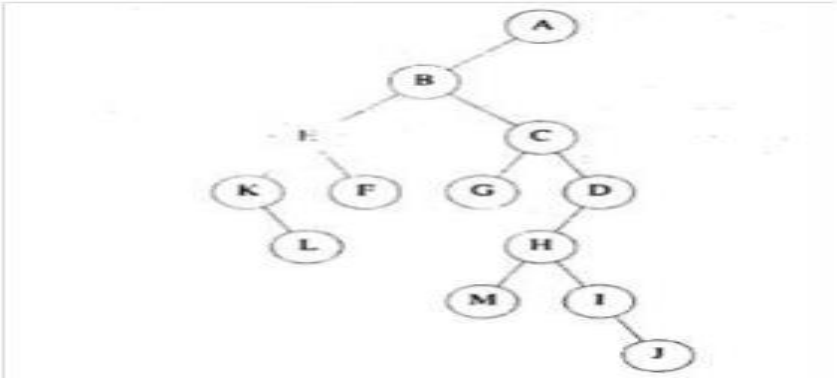



**Figure 16: Left Child-Right Child Representation**

Figure 17 shows the two additional examples of trees represented as left child-right sibling trees and as left child-right child trees. Left child-right child trees are also known as binary trees.

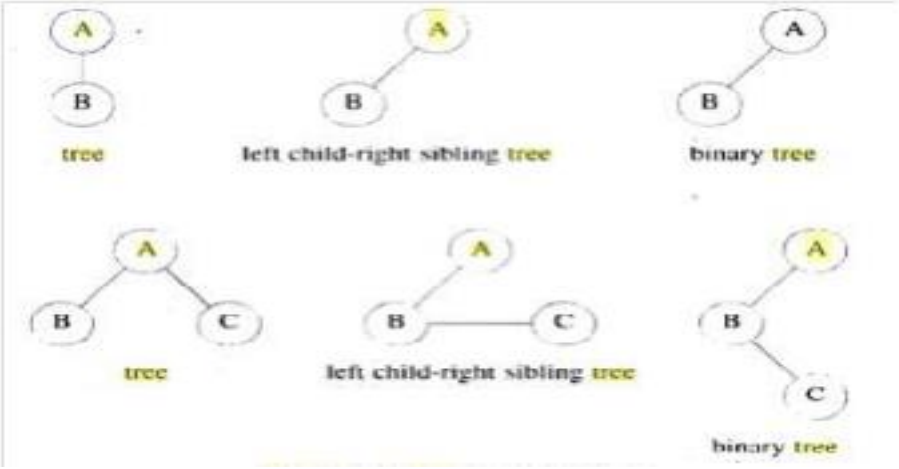



**Figure 17: Binary trees**

## 4.7 ITERATIVE INORDER TRAVERSAL

A node that has no action indicates that the node is added to the stack, while a node that has a printf action indicates that the node is removed from the stack (Figure 18). The left nodes are stacked until a null node is reached, the node is then removed from the stack, and the node's right child is stacked. The traversal continues with the left child. The traversal is complete when

the stack is empty. The output of iterative inorder traversal for expression tree is given below as **A/B*C*D+E**

```
void iter_inorder(tree_pointer node)
{
        int top= -1;      /* initialize stack */
        tree_pointer stack[MAX_STACK_SIZE];
        for (;;)
        {
                for (; node; node=node->left_child)
                add(&top, node);            /* add to stack */
                node= delete(&top);
                  /* delete from stack */
                if (!node) break;           /* empty stack */
                        printf("%D", node->data);
                node = node->right_child;
        }
}
```

**Figure 18: Inorder Iterative traversal**

**LEVEL-ORDER TRAVERSAL**

This traversal uses a queue (Figure 22). We visit the root first, then the root's left child followed by the root's right child. We continue in this manner, visiting the nodes at each new level from the leftmost node to the rightmost node. function addq( ) adds a tree node to queue and function deleteq( ) deletes the front tree node from the queue.

Level order traversal begins by adding the root to the queue,. the function operates by deleting the node at the front of the queue, printing out the node's data field, and adding the node's left and right children to the queue. since a node's children are at the next lower level, and we add the left child before the right child, the function prints out the nodes using the ordering scheme and thus level order traversal of arithmetic expression tree is **+*E*D/CAB**

```
void level_order(tree_pointer ptr)
{       /* level order tree traversal */
        int front = rear = 0;
        tree_pointer queue[MAX_QUEUE_SIZE];
        if (!ptr) return;          /* empty queue */
        addq(front, &rear, ptr);
        for (;;)
        {
                ptr = deleteq(&front, rear);
                if (ptr)
                {
                    printf("%d", ptr->data);
                    if (ptr->left_child)
                        addq(front, &rear, ptr->left_child);
                    if (ptr->right_child)
                        addq(front, &rear, ptr->right_child);
                }
                else
                    break;
        }
}
```

Figure 19: Level Order Traversal

**The Satisfiability Problem:-**

Consider the set of formulas that we can construct by taking variables x1,x2,x3,….xn and the operators ∧ (and), ∨ (or) and ¬ (not). The variables can hold only one of the 2 possible values, true or false. The set of expressions that we can form using these variables and operators is defined by the following rules:

1) A variable is an expression
2) If x and y are expressions, then ¬ x, x ∧ y, x ∨ y are expressions.
3) Parenthesis can be used to alter the normal order of evaluation, which is ¬ before ∧ before ∨.

These rules comprise the formulas in the prepositional calculus since other operations such as implication, can be expressed using ¬ , ∧ and ∨.

The expression: x1 ∨ (x2 ∧ ¬x3) is a formula (read as "x1 or x2 and not x3). If x1 and x3 are FALSE and x2 is TRUE, then the value of the expression is:

FALSE ∨ (TRUE ∧ ¬ FALSE)

= FALSE ∨ TRUE

= TRUE

The satisfiability problem for formulas of the prepositional calculus asks if there is an assignment of values to the variables that causes the value of the expression to be true. Assuming that a formula is already in a binary tree, say (x1 ∧ ¬x2) ∨ (¬x1 ∧ x3) ∨ ¬x3

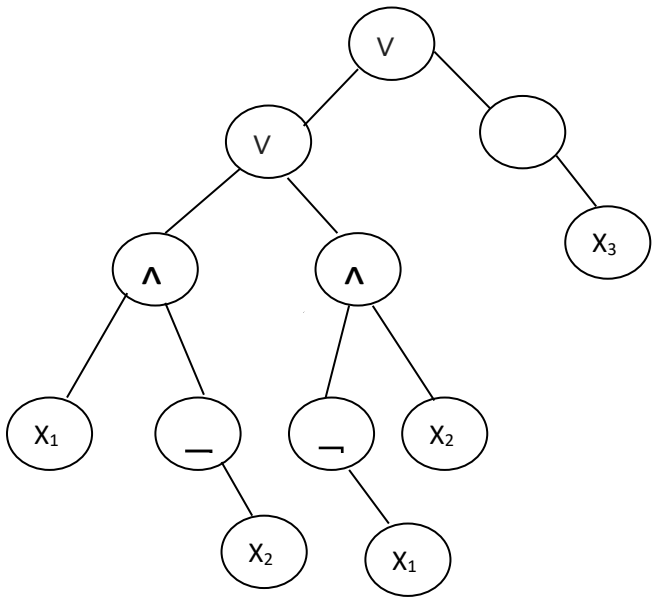



Figure 20: Satisfiability Problem

## 4.8 THREADED BINARY TREES

Shortcoming of linked representation of binary tree: There may be more null links than actual pointers. **A.J.Perlis and C.Thornton** have devised a way to make use of these null links. **Solution:** This drawback can be overcome by replacing null links by pointers, called **threads**, to other nodes in the tree.

To construct the threads, we use the following **rules:**

1) If ptr→leftChild =NULL, we replace ptr→leftChild with a pointer to the node that would be visited before ptr in an inorder traversal. i.e. we replace the null link with a pointer to the inorder predecessor of ptr (Figure 5.20).

2) If ptr→rightChild =NULL, we replace ptr→rightChild with a pointer to the node that would be visited after ptr in an inorder traversal. i.e. we replace the null link with a pointer to the inorder successor of ptr.

The node structure is given by following **C declarations**:

```
typedef struct threadedPointer node*;
typedef struct
{
        short int leftThread;
        threadedPointer leftChild;
        char data;
        threadedPointer rightChild;
        short int rightThread;
}threadedTree;
```

A empty threaded binary tree is represented by its header node as in figure 23

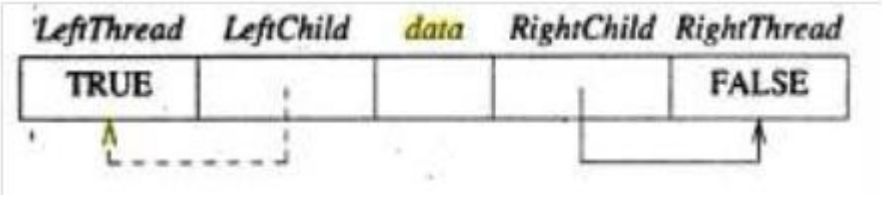
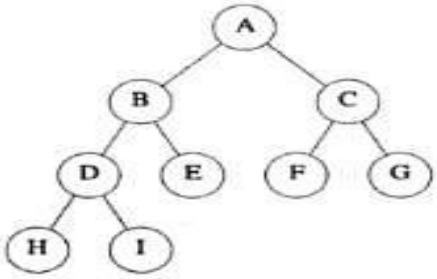



Figure 21: Empty node structure for threaded binary tree




(b)

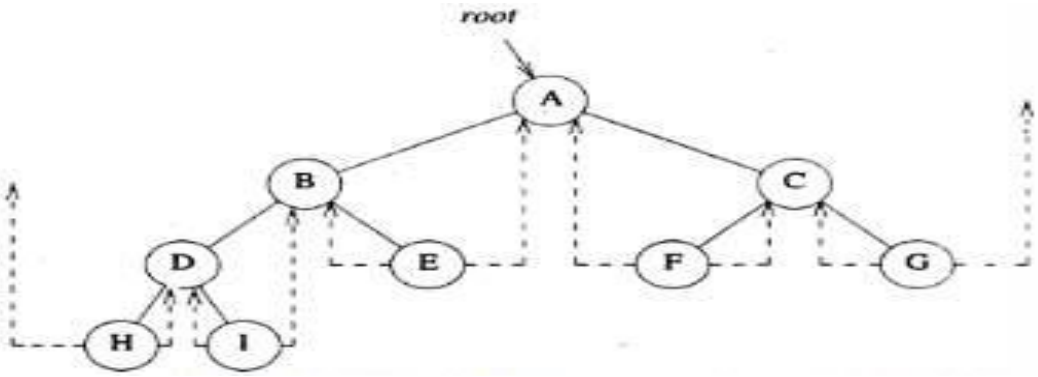



**Figure 22: Binary tree and its corresponding Threaded binary tree**

Figure 22 shows the binary tree with its new threads drawn in as broken lines. This tree has 9 nodes and 10 NULL links ,which have been replaced by threads. if we traverse the tree in inorder, the nodes will be visited in the order H,D,I,B,E,A,F,C,G. for example, node E has a predecessor thread that points to B and a successor thread that points to A

When we represent the tree in memory, we must be able to distinguish between threads and normal pointers. This is done by adding two additional fields to the node structure, leftThread and rightThread (Figure 22).

Assume that ptr is an arbitrary node in a threaded tree. If ptr->leftThread=TRUE, then ptr->leftChild contains a thread; otherwise it contains a pointer to the left child (Fig 24). Similarly, if ptr->rightThread=TRUE, then ptr->rightChild contains a thread; otherwise it contains a pointer to the right child (Figure 24).

In figure two threads have been left dangling: one in the left child of H , the other in the right child of G. We handle the problem of the loose threads by having them point to the header node, root. The variable 'root' points to the header as shown in figure 23.

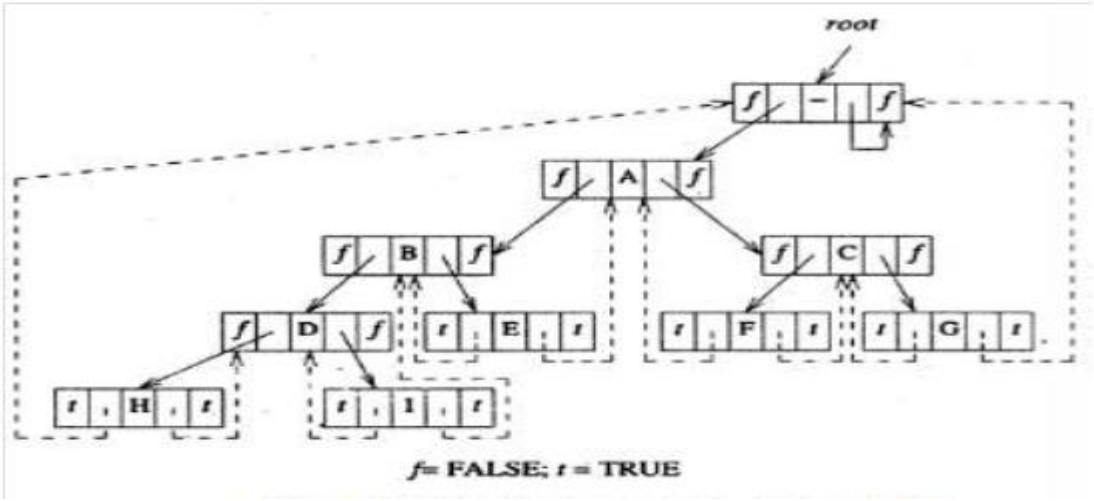



**Figure 23: Memory representation of threaded binary tree**

## 1. Inorder traversal of a threaded binary tree

```
threaded_pointer insucc(threaded_pointer tree)
{
        threaded_pointer temp;
        temp = tree->right_child;
        if (!tree->right_thread)
                while (!temp->left_thread)
                        temp = temp->left_child;
        return temp;
}
```

```
void tinorder(threaded_pointer tree)
{
        /* traverse the threaded binary tree inorder */
        threaded_pointer temp = tree;
        for (;;)
        {
            temp = insucc(temp);
            if (temp==tree) break;
                printf("%3c", temp->data);
        }
}
```

## 2. Inserting A Node Into A Threaded Binary Tree

Let new node 'r' be has to be inserted as the right child of a node 's' (Figure 25).

The cases for insertion are

1) If s has an empty right subtree, then the insertion is simple and diagrammed in fig 25 a.

2) If the right subtree of s is not empty, then this right subtree is made the right subtree of r after insertion. When this is done, r becomes the inorder predecessor of a node that has a leftThread==true field, and consequently there is a thread which has to be updated to point to r. The node containing this thread was previously the inorder successor of s. Figure 25b illustrates the insertion for this case.

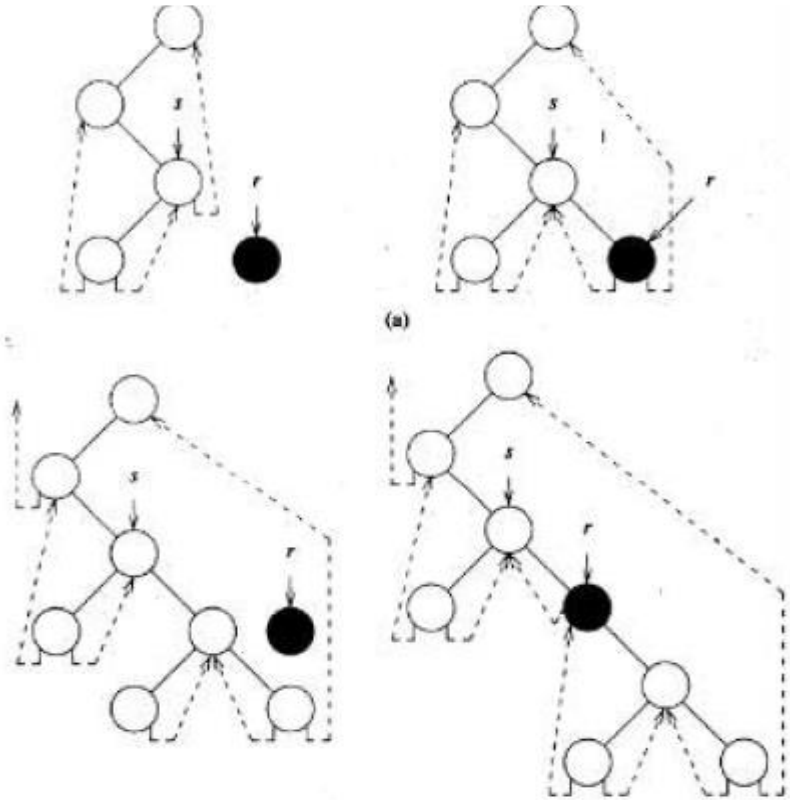



**Figure 25: Insertion of node into threaded binary tree**

**Insertion of node into threaded binary tree:-**

```
void insertRight(threadedPointer s,threadedPointer r)
        {
                r→rightChild=parent→rightChild;
                r→rightThread=parent→rightThread;
                r→leftChild=parent;
                r→rightThread=TRUE;
                s→rightChild= child;
                s→rightThread=FALSE;
                if(!r→rightThread)
                {
                        temp=insucc(r);
                        temp→leftChild=r;
                }
 }
```

## Advantages

1. By doing threading we neglect the recursive method of traversing a Tree, which makes use of stack and consumes many memory and time.
2. The node can keep record of its root.

## Disadvantages

1. Insertion and deletion operation become more difficult.
2. Tree traversal algorithm becomes difficult.
3. Memory required to store a node increase. Each node has to store the information whether the links is normal links or threaded links.

**4. Joining and splitting binary tree**

There are two types of join operation on a binary search tree:

1) **ThreeWayJoin(small,mid,big):** this creates a binary search tree consisting of the pairs initially in the BST small and big as well as pair mid. it is assumed that each key in small is smaller than mid.key and that each key in big is greater than mid.key. following the join , both small and big are empty.

2) **TwoWayJoin(small,big):** this joins the two binary search tree small and big to obtain a single BST that contains all the pairs originally in small and big. it is assumed that all keys of small are smaller than all keys of big and that following the join both small and big are empty.

3) **Split(theTree,k,small,big,mid)**

Splitting a binary search tree will produce three trees: small, mid and big.

- If key is equal to root->data, then root->llink is the small, root->data is mid & root->rlink is big.
- If key is lesser than the root->data, then the root's along with its right subtree would be in the big.
- If key is greater than root->data, then the root’s along with its left subtree would be in the small**.**